

2. ALGORITHMS AND COMPLEXITY

- Principles
- Efficiency analysis
- Complexity analysis
- O-notation
- Recursive algorithms

Principles (1)

- An **algorithm** is a step-by-step procedure for solving a stated **problem**.
- The algorithm will be performed by a **processor**.
 - The processor may be human, mechanical, or electronic.
- The algorithm must be expressed in **steps** that the processor is capable of performing.
- The algorithm must eventually **terminate**.
 - Otherwise it will never yield an answer.

Principles (2)

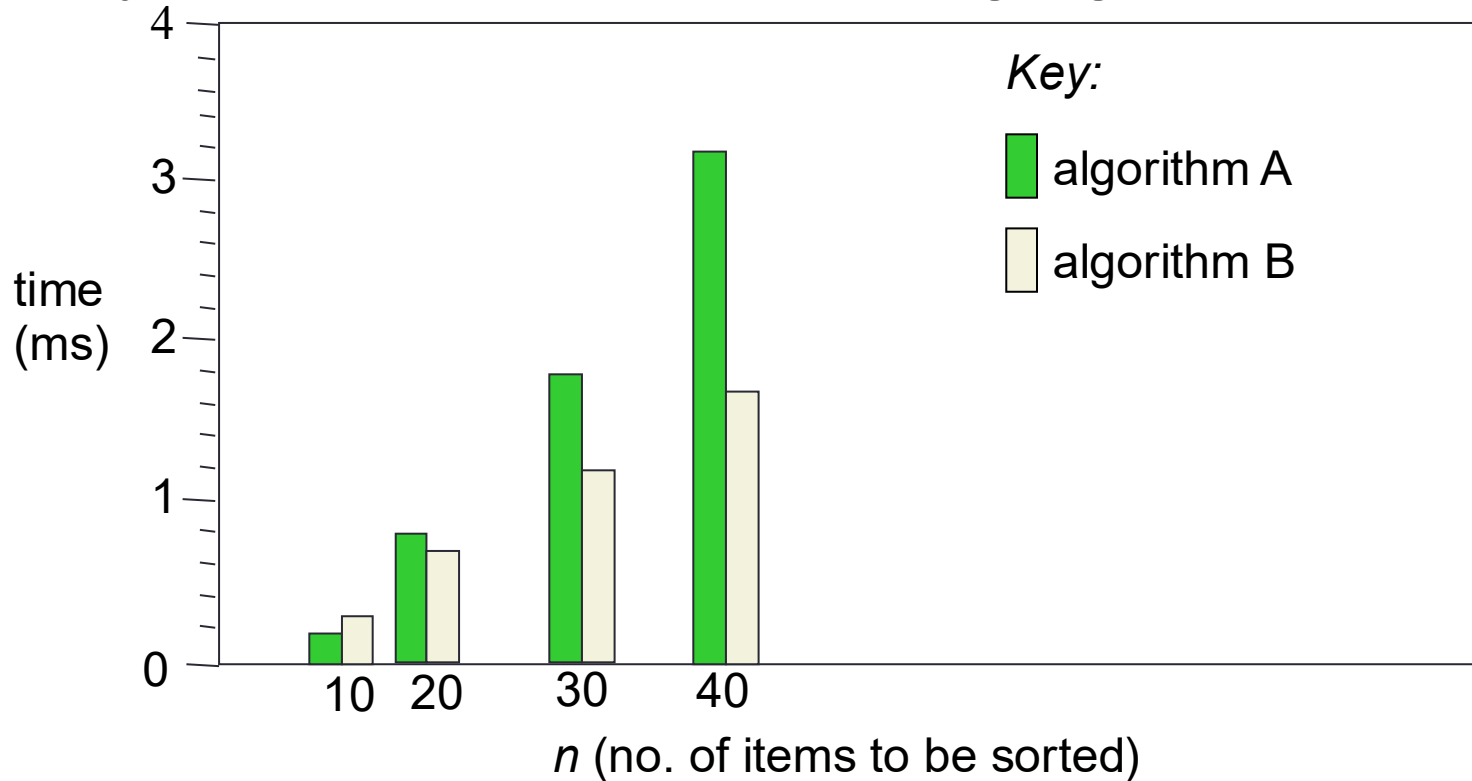
- The algorithm must be expressed in some **language** that the processor “understands”. (But the underlying algorithm is independent of the particular language chosen.)
- The stated problem must be **solvable**, i.e., capable of solution by a step-by-step procedure.
- Some interesting problems are unsolvable. E.g.:
 - Can we devise an algorithm to determine whether a given program will terminate or not? [i.e. a single algorithm for all program/input pairs]
 - No such algorithm exists. This is Turing’s famous “halting problem”.

Efficiency analysis

- Given a choice of algorithms to solve the same problem, which algorithm is “best”?
- Given a particular algorithm:
 - How much **time** does it require?
 - How much **space** (memory) does it require?
 - 可行的 • Is it **feasible** to use it at all?
In other words, is it efficient enough to be usable in practice?
- In general, both time and space requirements depend on the algorithm’s input data (usually the *size* of the input data).

Example: efficiency

- Hypothetical profile of two sorting algorithms:



- As n increases, A's time requirement grows faster than B's. So algorithm B is better!

How should we measure time?

- Measure time in seconds?
 - + useful in practice
 - dependent on programming language, compiler, processor speed.
- Count algorithm steps?
 - + not dependent on compiler or processor speed
 - dependent on the way the algorithm is expressed (few “big” steps vs many “small” steps).
- Count **characteristic operations**?
 - + dependent only on the algorithm itself
 - + good measure of the algorithm’s intrinsic efficiency.

e.g., arithmetic ops in numerical algorithms, or comparisons in search algorithms

Example: simple power algorithm (1)

- **Simple power algorithm:**

To compute b^n :

1. Set p to 1.
2. For $i = 1, \dots, n$, repeat:
 - 2.1. Multiply p by b .
3. Terminate yielding p .

Example: simple power algorithm (2)

- Analysis (counting multiplications):

Step 2.1 performs 1 multiplication.

This step is repeated n times.

No. of multiplications = n

To compute b^n :

1. Set p to 1.
2. For $i = 1, \dots, n$, repeat:
 - 2.1. Multiply p by b .
3. Terminate yielding p .

Approximate efficiency analysis

- For many interesting algorithms, the exact number of operations is too difficult to analyse mathematically.
- We can simplify the analysis by keeping the fastest-growing term but *neglecting all slower-growing terms*.

Example: smart power algorithm (1)

- Idea: $b^{1000} = b^{500} \times b^{500}$. If we know b^{500} , we can compute b^{1000} with only 1 more multiplication!

- **Smart power algorithm:**

To compute b^n :

1. Set p to 1, set q to b , and set m to n .
2. While $m > 0$, repeat:
 - 2.1. If m is odd, multiply p by q .
 - 2.2. Halve m (discarding any remainder).
 - 2.3. Multiply q by itself.
3. Terminate yielding p .

Example: smart power algorithm (2)

- Analysis (counting multiplications):

Steps 2.1–2.3 together perform *at most* 2 multiplications.

These steps are repeated as often as we must halve the value of n (discarding any remainder) until it reaches 0, i.e., approx. $\log_2 n + 1$ times (see maths revision!).

Ignoring the insignificant 1, max. no. of multiplications

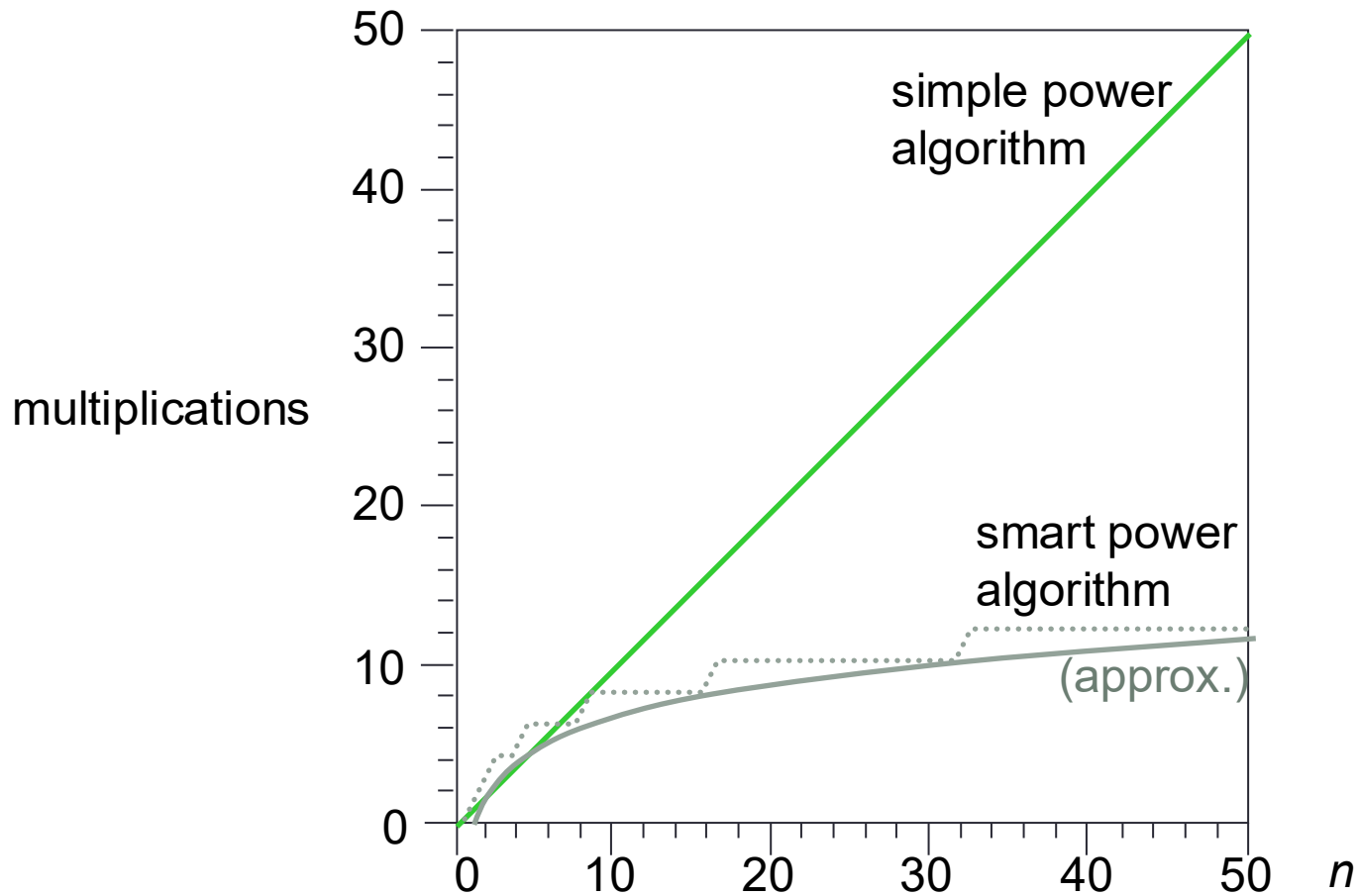
$$\approx 2 \log_2 n$$

To compute b^n :

1. Set p to 1, set q to b , and set m to n .
2. While $m > 0$, repeat:
 - 2.1. If m is odd, multiply p by q .
 - 2.2. Halve m (discarding any remainder).
 - 2.3. Multiply q by itself.
3. Terminate yielding p .

Example: power algorithms

- Comparison:



Complexity analysis

- We can further simplify the analysis, by *neglecting the constant factor in the fastest-growing term*.
- The resulting formula is the algorithm's **time complexity**. It focuses on the *growth rate* of the algorithm's time requirement.

Example: power algorithms complexity (1)

- Analysis of *simple* power algorithm (counting multiplications):

No. of multiplications = n

Time required $\approx n t_{\text{mult}}$ ----- where t_{mult} is the time per multiplication (a constant)
 which is proportional to n .

Time complexity is **of order n** .

This is written **$O(n)$** and referred to as “big o of n ”..

Example: power algorithms complexity (2)

- Analysis of smart power algorithm (counting multiplications):

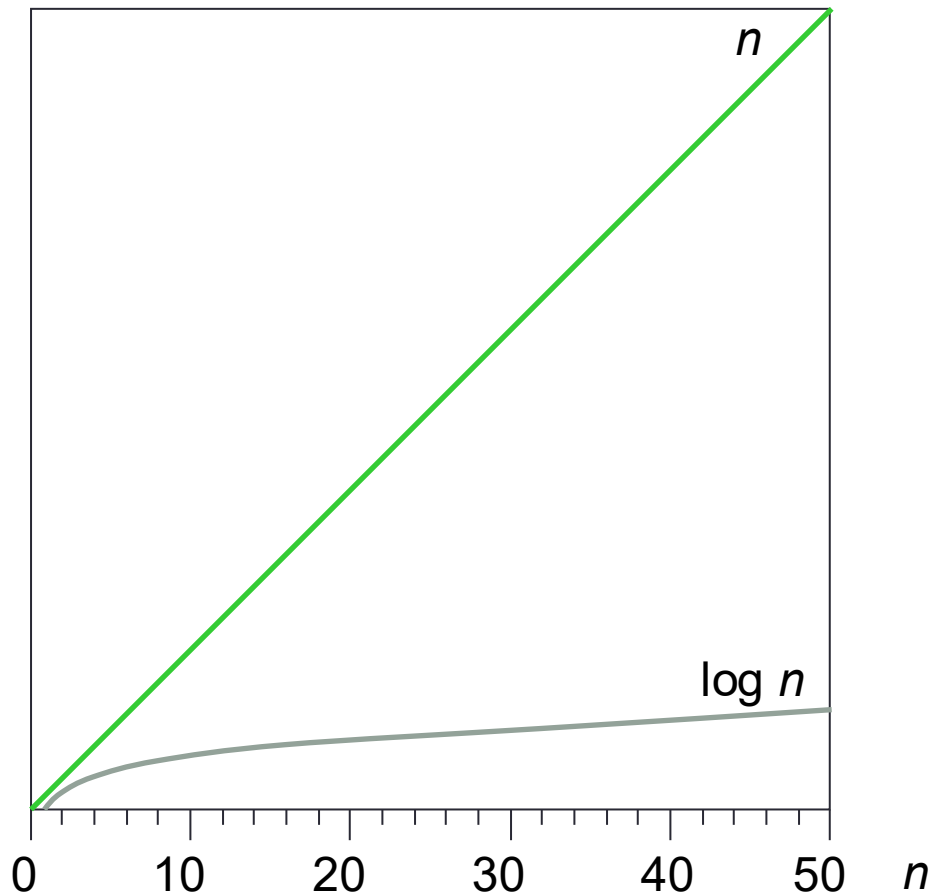
Max. no. of multiplications
 $\approx 2 \log_2 n$

Time required $\approx 2 t_{\text{mult}} \log_2 n$
which is proportional to $\log_2 n$.

Time complexity is **of order $\log_2 n$** .
This is written **$O(\log n)$** .

Example: power algorithms complexity (3)

- Comparison:



O-notation (1)

- We have seen that $O(\log n)$ signifies a slower growth rate than $O(n)$.

So an $O(\log n)$ algorithm is inherently better than an $O(n)$ algorithm – at least for large values of n .

- Complexity $O(X)$ means “of order X ”, i.e., growing proportionally to X .

Here X signifies the growth rate, *neglecting slower-growing terms and constant factors*.

O-notation (2)

- Common time complexities:

$O(1)$	constant time	(feasible)
$O(\log n)$	logarithmic time	(feasible)
$O(n)$	linear time	(feasible)
$O(n \log n)$	log linear time	(feasible)
$O(n^2)$ 二次的	quadratic time	(often feasible)
$O(n^3)$	cubic time	(less often feasible)
$O(2^n)$	exponential time	(rarely feasible)
$O(n!)$	factorial time	(more rarely feasible)

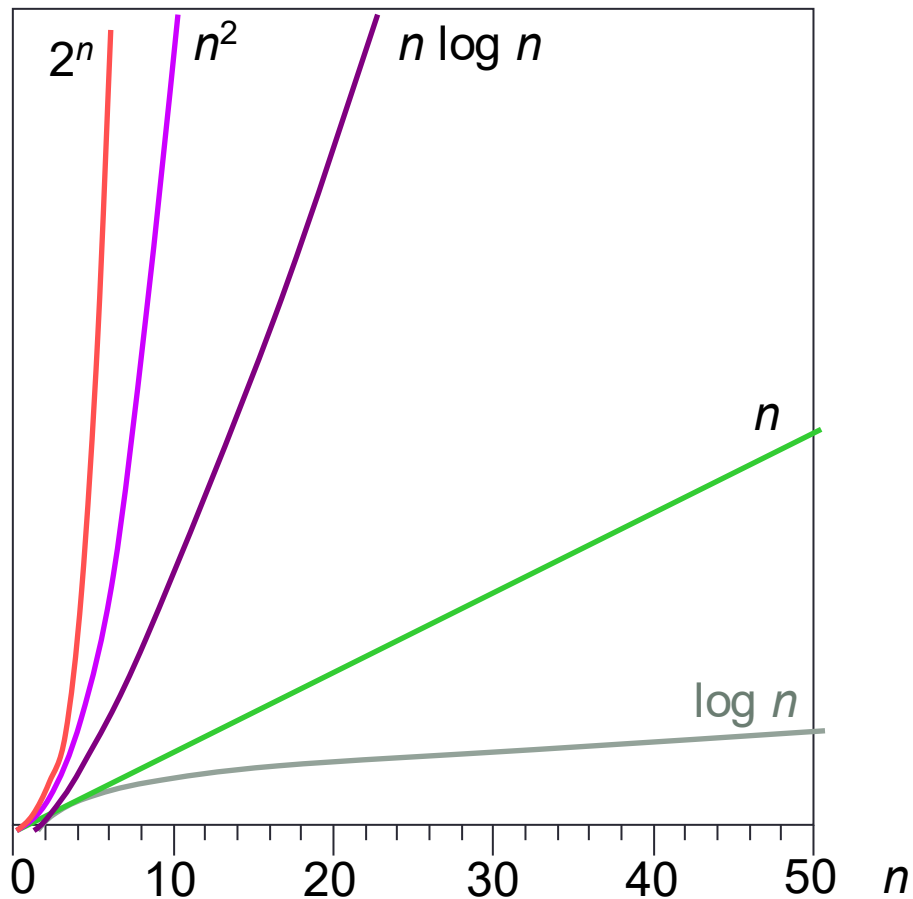
Complexity: growth rates (1)

- Numerical comparison of growth rates:

$\log n$	3.3	4.3	4.9	5.3
n	10	20	30	40
$n \log n$	33	86	147	213
n^2	100	400	900	1,600
n^3	1,000	8,000	27,000	64,000
2^n	1,024	1.0 million	1.1 billion	1.1 trillion
$n!$	3.4million	2.4 quintillion	265 nonillion	~1 quindecillion

Complexity: growth rates (2)

- Graphical comparison:



Recursive algorithms

- A **recursive algorithm** is an algorithm in which at least one step is a “call” to itself.
- Sometimes a problem can be solved *either* by an iterative algorithm *or* by a recursive algorithm. The recursive version tends to be:
 - + more elegant and easier to understand
 - less efficient (extra calls consume time and space).
- Sometimes a problem can be solved *only* by a recursive algorithm.

When does recursion work?

- Given a recursive algorithm, how can we sure that it terminates?
- The algorithm must have:
 - at least one “easy” case
 - at least one “hard” case.
- In an “easy” case, the algorithm must terminate without calling itself.
- In a “hard” case, the algorithm may call itself, but only to deal with an “easier” case.
- For some simple examples see ProgIT week 9 material.

Examples:

Factorial: calculate $n! = n \times (n-1) \times (n-2) \times \dots \times 1$

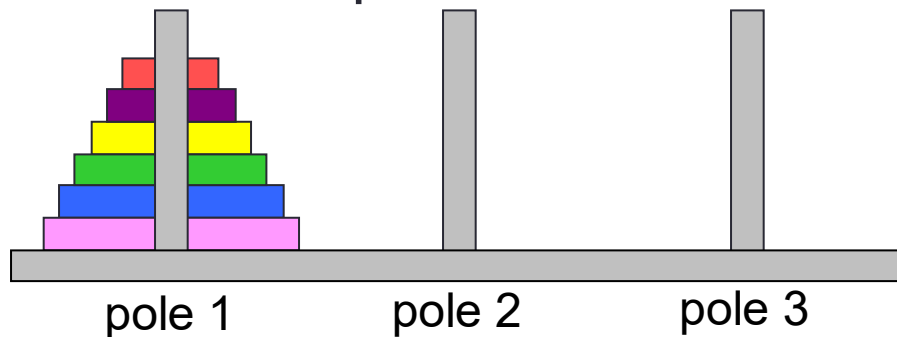
```
1    if n=1
1.1  return 1
2.   else
2.1  return n*factorial(n-1)
```

Power: calculate m^n

```
1.   if n=0 then
1.1  return 1
2.   else
2.1  return (m*power(m,n-1))
```

Example: Towers of Hanoi (1)

- Three vertical poles (1, 2, 3) are mounted on a platform.
- A number of discs are provided, all with different diameters. Each disc has a hole in its centre.
- All discs are initially threaded on to pole 1, forming a tower with the largest disc at the bottom and the smallest disc at the top.

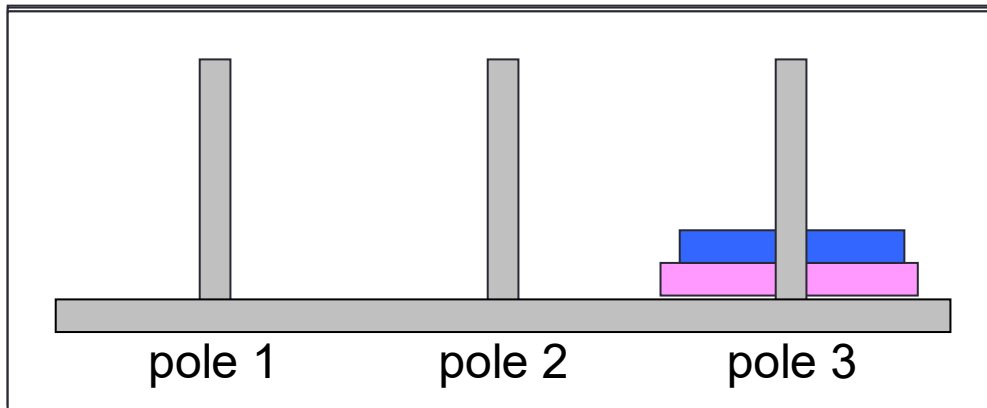


Example: Towers of Hanoi (2)

- Rules:
 - One disc may be moved at a time, from the top of one pole to the top of another pole.
 - A larger disc may not be moved on top of a smaller disc.
- Problem: Move the tower of discs from pole 1 to pole 3.

Example: Towers of Hanoi (3)

- Animation (with 2 discs):



Example: Towers of Hanoi (4)

- **Towers of Hanoi algorithm:**

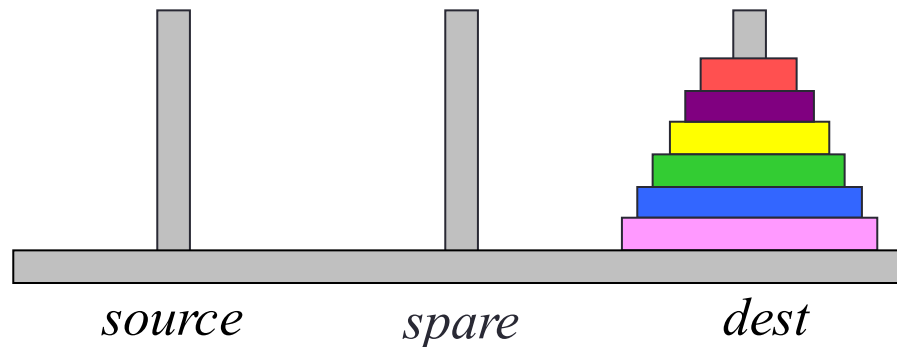
To move a tower of n discs from pole *source* to pole *dest*:

1. If $n = 1$:
 - 1.1. Move a single disc from *source* to *dest*.
2. Else, if $n > 1$:
 - 2.1. Let *spare* be the remaining pole, other than *source* and *dest*.
 - 2.2. Move a tower of $(n-1)$ discs from *source* to *spare*.
 - 2.3. Move a single disc from *source* to *dest*.
 - 2.4. Move a tower of $(n-1)$ discs from *spare* to *dest*.
3. Terminate.

Example: Towers of Hanoi (5)

- Animation (with 6 discs):

1. If $n = 1$:
 - 1.1. Move a single disc from *source* to *dest*.
2. If $n > 1$:
 - 2.1. Let *spare* be the remaining pole, other than *source* and *dest*.
 - 2.2. Move a tower of $(n-1)$ discs from *source* to *spare*.
 - 2.3. Move a single disc from *source* to *dest*.
 - 2.4. Move a tower of $(n-1)$ discs from *spare* to *dest*.
3. Terminate.



Example: Towers of Hanoi (6)

- Analysis (counting moves):

Let the total no. of moves required to move a tower of n discs be $moves(n)$. Then:

$$moves(n) = 1 \quad \text{if } n = 1$$

$$moves(n) = 1 + 2 \, moves(n-1) \quad \text{if } n > 1$$

Solution:

$$moves(n) = 2^n - 1$$

Time complexity is $O(2^n)$.

Wait ... what?

I've missed out the proof.

See additional (optional) slides on Moodle on Tower of Hanoi – shows why the recursion works and proves this step (for interest, wouldn't expect you to do it!)